Bayesian Belief Network-Driven Proactive Fault Tolerance for Microservice Placement in Fog Computing Environments

Rahul Akshath Rajendran
3122235002095
rahul2310405@ssn.edu.in

Mohit R
3122235002072
mohit2310893@ssn.edu.in

Dr. Sabireen H, Assistant Professor
Project Guide
sabireenh@ssn.edu.in

Department of Information Technology
SSN College of Engineering, Kalavakkam, Chennai 603110, India

Abstract

Fog computing extends cloud services to the network edge in order to satisfy the stringent latency requirements of Internet-of-Things (IoT) applications such as remote cardiac monitoring, intelligent video surveillance, and immersive gaming. The fog nodes that host these applications are, however, heterogeneous, resource-constrained, and considerably less reliable than data-centre hardware, and the mechanisms that protect them are predominantly reactive: recovery begins only after a node has failed. The few proactive schemes in the literature migrate on a fixed threshold over a single metric. This paper presents and evaluates a Bayesian Belief Network (BBN) driven proactive fault-tolerance framework for microservice placement. A causal BBN, extended with telemetry trends and device-class conditioning and trained by maximum a posteriori expectation-maximisation, infers from whatever telemetry is available the probability that each fog node fails within the next two decision epochs, and names the latent condition behind that risk. A controller built inside iFogSim2 converts the posterior into risk-aware placement, expected-loss migration, and criticality-aware selective replication. On operating regimes and random seeds withheld from training, the BBN reaches a ROC-AUC of 0.866 and a PR-AUC of 0.639 at a failure prevalence of 0.26%, level with the strongest of eight alternative predictors, while producing the best-calibrated probabilities (Brier score 0.00128, expected calibration error 0.0005). Across 25 paired closed-loop runs, the framework cuts the downtime caused by failures and migrations by 97%, 85%, and 55% relative to no fault tolerance, reactive checkpointing, and threshold-triggered migration, misses the 150 ms deadline on 0.13% of critical alarms, and holds a warm standby for only 0.2% of sessions. With 30% of telemetry channels missing, more often on degrading nodes, it incurs 29% less fault-related downtime than the same controller driven by gradient boosting.

Index Terms

Bayesian belief network, fog computing, proactive fault tolerance, microservice placement, reliability, iFogSim2, expectation-maximisation, probability calibration, Internet of Things.

I. Introduction

THE proliferation of latency-sensitive Internet-of-Things (IoT) applications has exposed a structural limitation of the centralised cloud model. An electrocardiogram stream that must be classified within a few tens of milliseconds, or a surveillance feed that must trigger a pan-tilt-zoom actuator before a tracked object leaves the frame, cannot tolerate the round-trip delay to a distant data centre. Fog computing addresses this by interposing a tier of computational resources—gateways, routers, base stations, and micro data centres—between the end devices and the cloud, so that application modules execute within one or two network hops of the sensors that feed them [1], [2].

This proximity is purchased at the cost of reliability. Whereas a cloud data centre offers redundant power, cooling, and network paths, a fog node is frequently a commodity device deployed in an uncontrolled environment, sharing a constrained uplink, operating on limited or battery power, and serving mobile users whose handovers continually reshape the load it carries. Empirical studies of distributed infrastructure consistently report that such nodes fail far more often than data-centre hosts, and that failures are rarely instantaneous: a node typically degrades through a period of rising utilisation, thermal stress, queue

build-up, and link deterioration before it becomes unresponsive [3]. That degradation window is precisely the opportunity that a proactive mechanism should exploit.

The fog fault-tolerance literature has, however, largely declined to exploit it. The dominant mechanisms—dynamic checkpointing [4], acknowledgement-based retransmission [5], and heartbeat monitoring with failover to a replicated backup [6], [8]—are reactive by construction. They detect the absence of a response or the loss of a message, and then initiate recovery. For a best-effort workload this is adequate; for a cardiac arrhythmia classifier it means that the very interval during which an alarm should have been raised is consumed by state restoration. Even the schemes that describe themselves as proactive reduce the decision to a single scalar comparison, such as creating a checkpoint whenever CPU utilisation exceeds twenty per cent [4], or migrating a microservice when a composite health index falls below a preset fault threshold [8]. A fixed threshold on a single metric is simultaneously too eager and too late: it fires on transient load spikes that resolve without incident, and it stays silent when a node is degrading along a dimension the metric does not observe.

What is missing is a principled representation of uncertainty. Fog node health is a latent condition, inferred indirectly from several noisy and mutually dependent indicators, some of which may be unavailable at any given instant because a telemetry agent has itself been affected. Bayesian Belief Networks (BBNs) are the canonical formalism for exactly this situation. A BBN encodes causal structure as a directed acyclic graph and quantifies it with conditional probability tables (CPTs), supports exact inference of a posterior over an unobserved variable given whatever evidence happens to be available, degrades gracefully when evidence is partial by marginalising over the missing variables, and permits expert knowledge to be combined with observed data [9]. BBNs have been applied with documented success to software defect and reliability prediction [10], to dependability, risk analysis, and maintenance in industrial systems [11], and to environmental modelling under sparse observation [12]. Their transfer to fog reliability management is natural and, to the best of our knowledge, unexplored.

This paper presents a proactive fault-tolerance framework in which a BBN estimates, for every fog node, the probability that it will fail within a look-ahead horizon, and in which that posterior drives placement, migration, and redundancy decisions. The design was proposed at Review 2; this paper reports its implementation inside the iFogSim2 simulator and its evaluation on operating conditions never seen in training. The contributions are fourfold:

- **A BBN-based failure-prediction engine** that infers the posterior probability of imminent node failure from heterogeneous telemetry, marginalises over channels that fail to report instead of imputing them, and names the latent condition responsible for each prediction. Its tables are learned from simulated telemetry by maximum a posteriori expectation-maximisation over compact ranked-node parameterisations, and refined online from observed outcomes.
- **A risk-aware placement and expected-loss migration policy** in which the posterior enters a multi-objective utility over latency, energy, and risk, and in which every session migrates at its own threshold, derived from the relative cost of failing and of moving, rather than at a global fault threshold.
- **A criticality-aware selective replication scheme** that chooses, per session, the protection tier with the least overhead plus expected loss, so that standby capacity follows risk and consequence instead of being reserved uniformly.
- **An evaluation protocol designed to make these claims checkable**: whole operating regimes and random seeds are held out, a leak audit confirms that held-out failures are independent of training failures, an accuracy ceiling bounds what any predictor could achieve, and closed-loop comparisons share their randomness and are tested pairwise.

The remainder of the paper is organised as follows. Section II surveys reactive and proactive fault tolerance in fog computing and the use of BBNs in reliability engineering. Section III presents the framework, its block diagram, and its workflow; Section IV details the algorithms and technologies used; Section V describes the dataset; and Section VI defines the performance metrics. Section VII describes the implementation, Section VIII reports the experiments, Section IX draws inferences from them, and Section X concludes.

II. Literature Survey

A. Reactive Fault Tolerance in Fog Environments

Rajab and Younis [4] propose a dynamic fault-tolerance-aware scheduling approach for healthcare systems on fog computing. Their scheme combines Dynamic Weighted Round Robin (DWRR) scheduling, in which each fog server is assigned a weight derived from its current CPU and memory utilisation, with primary-backup replication and dynamic checkpointing. Rather than checkpointing at fixed intervals, the system creates a checkpoint whenever CPU utilisation crosses a threshold of 0.2, and every checkpoint is replicated to a designated backup server so that execution can resume from the most recent saved state if the primary fails. The work demonstrates a clear throughput benefit from load-aware distribution and a reduced re-execution cost from adaptive checkpointing. Its limitations are equally clear: the threshold is fixed and workload-independent, the backup server consumes storage whether or not a failure ever occurs, evaluation is confined to a healthcare scenario, and the system possesses no capacity to anticipate a failure before the threshold condition is met.

Syed *et al.* [5] address reliability in vehicular networks through QAFT-SDVN, which integrates software-defined networking with a cloud-fog substrate. A centralised SDN controller classifies each message by type, deadline, and size, assigns a priority accordingly, and routes safety-critical traffic such as ambulance alerts to the nearest fog node while queueing non-safety traffic at lower priority. Fault tolerance is provided by acknowledgement-based retransmission: the sender awaits an ACK and

retransmits if none arrives within the expected interval. The approach roughly halves response time for prioritised traffic and reduces the message failure ratio, but recovery is purely reactive and consists solely of retransmission. No backup execution, checkpointing, or replica management is provided, the additional ACK traffic inflates network load, and the design is specialised to the vehicular domain.

Naim, Zain, and Jalil [6] place a fog middleware between users and application servers to improve availability. The middleware polls the primary server with heartbeat requests, maintains a replicated backup containing the same application and data, and on detecting the absence of a heartbeat response activates the backup and transparently redirects all subsequent user requests to it. A prototype recovered from three of four simulated failure scenarios without manual intervention. The mechanism is nevertheless strictly detection-driven: it waits until the server has already stopped responding, considers only a single backup so that a correlated double failure is unrecoverable, imposes continuous heartbeat overhead, and performs no load-aware server selection.

B. Optimisation-Based and Threshold-Based Proactive Placement

Reyana *et al.* [7] propose QoS-EO, a hybrid task-placement approach that combines Particle Swarm Optimisation with the Artificial Bee Colony algorithm. Candidate solutions encode task-to-node assignments and are scored by a fitness function over response time, energy consumption, reliability, throughput, and execution cost; PSO drives the swarm towards better assignments while ABC's worker, onlooker, and scout phases refine and diversify the search. Better placement reduces node overload and thereby improves fault tolerance indirectly. The cost is computational: running two metaheuristics in sequence is expensive, performance is sensitive to inertia-weight and learning-factor settings, and the search space grows unfavourably as the number of IoT devices increases. Crucially, reliability enters only as a static term in a fitness function; the scheme does not estimate the probability that any particular node will fail.

Gupta and Singh [8] come closest to a proactive design. Their Fault-Tolerance Aware Placement Algorithm (FTAPA) selects a primary fog node using a Dynamic Connections Load Balancer that evaluates active connections, resource availability, and node health; it simultaneously pre-selects a backup node, monitors the health of the primary against a computed fault threshold, and migrates the microservice to the backup before the primary fails outright. This anticipatory migration measurably reduces service downtime relative to detection-driven recovery. The authors themselves identify the residual weaknesses: backup nodes consume memory and processing capacity even when failures never materialise, continuous health monitoring adds overhead, and the fault threshold is difficult to set—too low and migration happens too late, too high and the system migrates unnecessarily. The threshold is a point estimate of a quantity that is inherently uncertain, and no confidence is attached to it.

C. Bayesian Belief Networks for Reliability Prediction

A Bayesian Belief Network is a directed acyclic graph whose vertices are random variables and whose edges encode direct probabilistic dependence, quantified by a conditional probability table for each vertex given its parents. The joint distribution factorises as the product of these local conditionals, which makes exact posterior inference tractable for the sparse structures that arise when causal knowledge is available. The foundational treatment by Pearl [9] established both the semantics of such networks and the message-passing algorithms that make inference practical.

The formalism has been applied extensively to reliability. Fenton, Neil, and Marquez [10] use BNs to predict software defects and reliability, arguing that the ability to combine causal process factors with quantitative measures, and qualitative expert judgement with observed data, overcomes well-known limitations of classical metrics-based models; the approach has been deployed by Motorola, Siemens, and Philips with reported accuracy. Weber *et al.* [11] survey a decade of BN applications to dependability, risk analysis, and maintenance, and document a sustained growth in adoption attributable to advantages over Markov chains, fault trees, and Petri nets—specifically the capacity to model complex systems with heterogeneous variables, to perform both predictive and diagnostic reasoning, and to update beliefs incrementally as evidence arrives. Aguilera *et al.* [12] review BNs in environmental modelling, a domain characterised, like fog telemetry, by sparse and unreliable observation, and emphasise the graceful handling of missing data. More recently, Pang and Abramov [13] analyse reliability and parameter selection for IoT communication using deep learning, confirming the importance of predictive reliability modelling for IoT while also illustrating the interpretability cost of black-box predictors—a cost that a BBN does not impose.

Two properties make BBNs particularly appropriate for fog node health. First, inference is well-defined under partial evidence: unobserved variables are marginalised out, so a prediction is still produced when a telemetry channel is unavailable, which is precisely the condition likely to coincide with impending failure. Second, the network is diagnostic as well as predictive: having inferred a high failure probability, the same model identifies which parent variables are most responsible, yielding an explanation that an operator can act on and that a threshold comparison cannot provide.

D. Comparative Analysis

Table I positions the five surveyed mechanisms against the framework presented in this paper.

TABLE I: Comparative Analysis of Fault-Tolerance Mechanisms in Fog Computing

Reference	Core Mechanism	Paradigm	Limitations Identified
Rajab & Younis (2021) [4]	DWRR scheduling, primary-backup replication, CPU-threshold checkpointing	Reactive	Fixed workload-independent threshold; standby storage consumed whether or not a failure occurs; cannot anticipate failure.
Syed et al. (2022) [5]	SDN priority routing, ACK-based retransmission	Reactive	Recovery is retransmission only; no replication or checkpointing; extra ACK traffic; vehicular-specific.
Naim et al. (2016) [6]	Heartbeat polling, replicated backup activation	Reactive	Detection-driven; single backup cannot survive correlated failure; heartbeat overhead.
Reyana et al. (2023) [7]	Hybrid PSO-ABC placement optimisation	Preventive (optimisation)	Computationally heavy; reliability is a static fitness term; no per-node failure probability.
Gupta & Singh (2025) [8]	DCLB selection, threshold-triggered proactive migration	Proactive (heuristic)	Threshold hard to set; standby reservation wastes capacity; prediction carries no confidence.
This Work	Causal BBN posterior inference, expected-loss migration, risk-tiered replication	Proactive (probabilistic)	Evaluated in simulation under a synthetic degradation fault model; failures without precursors remain unpredictable.

E. Inferences Drawn from the Survey

Read together rather than individually, the five systems support six inferences that shaped the design of this work.

1) **Fault tolerance in fog computing is overwhelmingly reactive.** Three of the five systems act only once a fault has been detected—by a missing heartbeat [6], an unacknowledged message [5], or a threshold breach during execution [4]—so the recovery latency is incurred at precisely the moment the application can least absorb it.
2) **Acting before failure demonstrably helps.** FTAPA [8] reports measurably reduced service downtime from migrating a microservice ahead of a predicted failure. The principle is sound, and this work builds on it rather than disputing it.
3) **Where prediction is attempted it is deterministic and single-metric.** Both Rajab and Younis [4] and Gupta and Singh [8] reduce the decision to a comparison of one scalar against a preset threshold, and the authors of both report the difficulty of choosing that threshold as an open limitation.
4) **Node health is treated as directly observable.** No surveyed system represents it as a latent condition that must be inferred from indicators which are noisy, mutually dependent, and sometimes absent—despite the fact that a telemetry channel is most likely to fall silent exactly when the node hosting it is degrading.
5) **Redundancy is provisioned uniformly rather than by risk.** A backup is reserved per task [4], per server [6], or per microservice [8], consuming standby capacity irrespective of whether a failure is likely; Gupta and Singh [8] identify this as a standing cost of their own design.
6) **Reliability enters optimisation only as a static term.** QoS-EO [7] scores reliability within a fitness function but never estimates the probability that a particular node will fail within a particular interval, so its reliability term cannot respond to changing conditions.

Taken together, these inferences point to a single conclusion: what the field lacks is a probabilistic and explainable estimate of imminent node failure, together with a decision rule that spends migration and redundancy in proportion to that estimate.

F. Selection of the Modelling Approach

Several established formalisms could in principle supply such an estimate, and the choice among them was made against four requirements imposed by the problem: the method must function without a large corpus of labelled failures, since none exists for fog topologies; it must continue to produce an estimate when part of the telemetry is unavailable; it must emit a calibrated probability, because the decision rules of Section III multiply that probability by a cost; and it must be interpretable, so that an operator can act on the reason for a prediction rather than only its magnitude. Table II summarises the candidates against these requirements.

TABLE II: Selection of the Modelling Approach

Candidate Methodology	Requirement Match	Assessment
Fixed Threshold Rules	Inadequate	Fail the first three requirements outright: no uncertainty, no partial-evidence reasoning, no calibrated output.
Metaheuristic Placement (PSO/ABC)	Inadequate	Optimises an assignment but never estimates the probability of failure at all.
Markov Chains / Fault Trees	Partially Adequate	State-space explosion at this system size; accommodate partial evidence poorly.
Discriminative Machine Learning	Partially Adequate	Needs abundant labelled data; must impute absent channels; confidence scores often poorly calibrated.
Bayesian Belief Network	Fully Adequate	CPTs seeded from priors and refined from data; marginalises over unobserved variables; formal posterior; causal and interpretable.

Fixed threshold rules fail the first three requirements outright. Metaheuristic placement optimises an assignment but never estimates failure at all. Markov chains and fault trees, the classical instruments of reliability analysis, suffer state-space explosion on systems of this size and accommodate partial evidence poorly—precisely the comparison Weber *et al.* [11] draw in concluding that Bayesian networks have displaced them across dependability and maintenance applications. Discriminative machine learning, including the deep-learning approach that Pang and Abramov [13] apply to IoT reliability, achieves high accuracy where abundant labelled data exists, but requires an input value in every feature slot and so must impute the very channels whose absence is diagnostic; the confidence scores of modern classifiers are also known to be miscalibrated [14], which matters when the output is to be multiplied by a cost.

A Bayesian Belief Network satisfies all four requirements simultaneously. Its conditional probability tables can be initialised from expert judgement and published reliability priors and then refined from observation, so it is usable from the first epoch. Inference marginalises over unobserved variables, so partial telemetry degrades the sharpness of the posterior without preventing a decision. The output is a probability in the formal sense, suitable for composition with a loss function. And because the graph encodes causal structure, the same inference that produces the posterior also identifies the parent configuration most responsible for it. This combination, rather than any single property, is why the Bayesian Belief Network was selected—and Section VIII tests each of these four claims against eight alternatives rather than taking them on trust.

G. Learning Bayesian Networks from Scarce and Incomplete Data

Two strands of the BBN literature became necessary once the network had to be learned rather than only elicited. The first concerns the size of conditional probability tables. A latent condition with five evidence parents of eight states each has 32,768 parent configurations per device class, almost none of which a failing node ever visits. Fenton, Neil, and Caballero [15] introduced *ranked nodes* for exactly this growth: each parent state maps to a point on [0, 1], the child is a truncated normal around a weighted combination of those points, and the table is generated rather than enumerated. The number of parameters then grows linearly with the parents instead of exponentially.

The second concerns the latent conditions themselves, which are never observed and so cannot be counted. Expectation-maximisation [16] replaces the missing counts by their expectation under the current parameters and re-estimates the parameters from those expected counts; with Dirichlet priors on the tables the procedure becomes maximum a posteriori estimation, which keeps rarely visited configurations close to their expert values [17]. Continuous telemetry must also be discretised. Supervised discretisation, which places cut points where they carry the most information about the class rather than at fixed quantiles, has long been known to outperform unsupervised binning for classification [18]; it matters here because failure precursors live in the extreme tail of each channel. Finally, field studies of large production systems document that hardware often degrades into a *fail-slow* state—still responding, but markedly slower—before it stops [19], which is the behaviour the fault model of Section V reproduces.

H. Research Gap

Table I makes the gap explicit. Every surveyed fault-tolerance mechanism either reacts to a fault that has already occurred, or anticipates it using a deterministic comparison against a fixed threshold on a small number of directly observed metrics. None represents node health as a latent variable, none quantifies the uncertainty of its own prediction, none continues to reason when telemetry is incomplete, and none uses a predicted failure probability to modulate how much redundancy a given module deserves. Conversely, the BBN literature provides a mature apparatus for probabilistic reliability inference under uncertainty, but has not been applied to the fog placement problem. This paper addresses that intersection.

III. Proposed Methodology

A. Block Diagram of the Framework

Fig. 1 presents the framework as four blocks connected by seven numbered flows. Block A is the fog environment, realised in iFogSim2: a cloud data centre for archival storage, 118 fog gateways that each host an electrocardiogram (ECG) analyser, and 20 to 80 patients' edge devices that send vital signs upward and receive alarms in return while their owners move along recorded mobility traces. Two components are added to the stock simulator. A telemetry agent on every node reports thirteen measurement channels once per decision epoch, and a fault injector wears nodes out under stress and eventually fails them. Block B is offline learning: the reports of 31 labelled simulation runs form a telemetry dataset, which is discretised and used to train the BBN by expectation-maximisation. Block C is the proactive fault-tolerance controller, which runs inside the simulator once per epoch and contains the three novelties: N1 infers the failure posterior, N2 places and migrates sessions, and N3 sizes their redundancy. Block D evaluates the result, comparing the BBN with eight other predictors and the controller with six other policies on runs that training never saw.

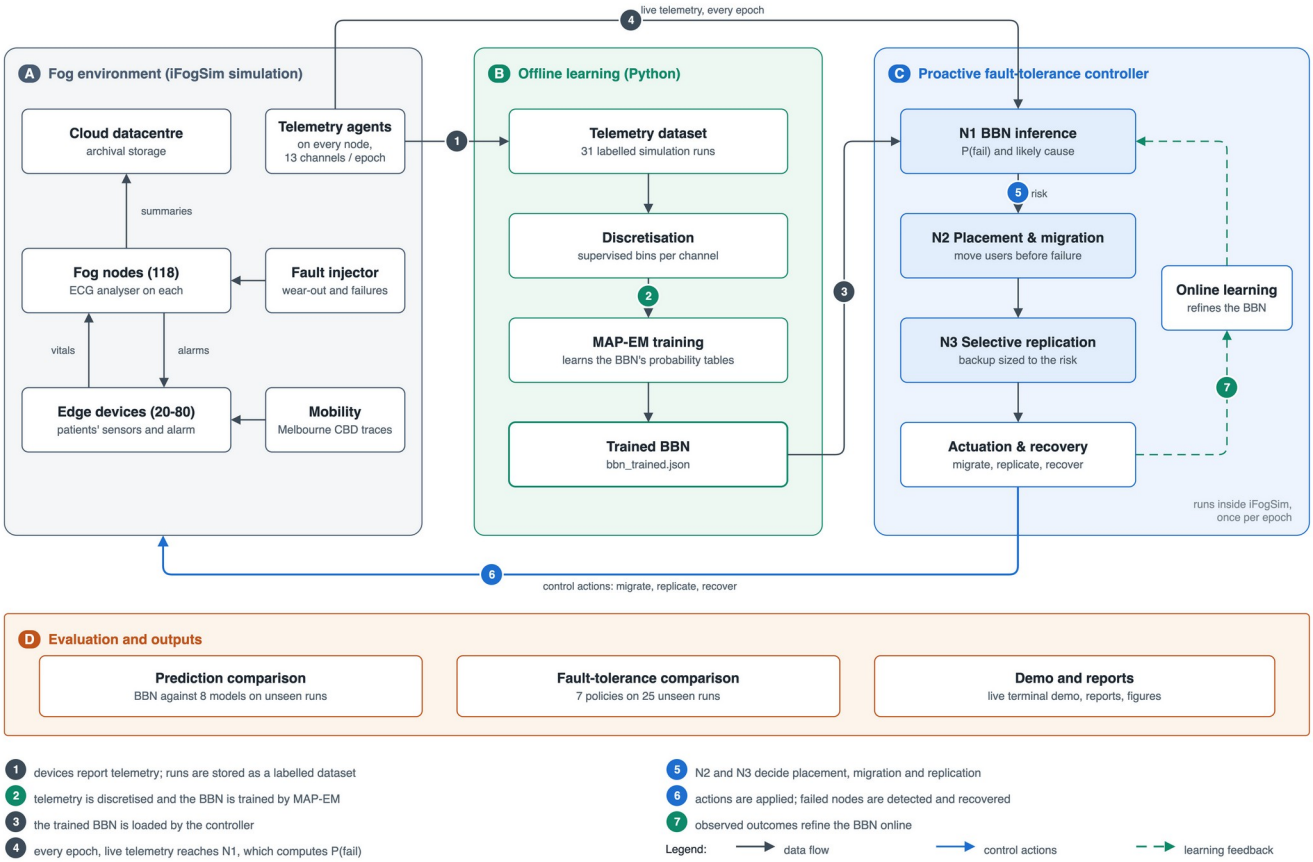


Fig. 1: Block diagram of the BBN-driven proactive fault-tolerance framework. (A) the simulated fog environment, (B) offline learning of the BBN, (C) the controller with its three novelties N1–N3, and (D) evaluation. Numbered flows: (1) telemetry is stored as a labelled dataset, (2) the BBN is trained, (3) the trained model is loaded by the controller, (4) live telemetry reaches N1 every epoch, (5) N2 and N3 act on the posterior, (6) actions are applied and failures recovered, (7) observed outcomes refine the BBN online.

The flows describe how one piece of information becomes a decision:

1) **Telemetry to dataset.** Every device reports its channels each epoch; the reports of complete runs, labelled with whether the node failed within the following two epochs, are written to disk.
2) **Dataset to trained BBN.** Training telemetry is cut into ordered bins chosen for their information about failure, and the network's tables are learned by MAP expectation-maximisation.
3) **Model to controller.** The trained network is exported as a single file that the controller inside the simulator loads; the same file drives the offline evaluation.
4) **Live inference.** At each epoch the controller reads the latest report of every fog node and computes P(fail within two epochs | evidence), together with the latent condition behind it.
5) **Decisions.** The posterior enters the placement utility and the migration rule (N2) and the choice of protection tier (N3).

6) **Actuation.** Migrations, replicas, and checkpoints are applied to the simulated fog; when a node fails regardless, the failure is detected and each affected session recovers as its tier allows.
7) **Feedback.** Once a report's horizon has elapsed its outcome is known, and the outcome updates the failure table of the running network.

The controller is logically centralised. In a deployment it would sit with the fog orchestration node at the proxy tier, where it sees an entire cluster of gateways without the round-trip delay of a cloud-hosted controller; in the simulator it is a CloudSim entity with the same view.

B. Novelty 1: BBN-Based Failure Prediction Engine

The core of the framework is a Bayesian Belief Network over fog-node condition, whose structure is shown in Fig. 2. The network is organised in three tiers. Observable evidence variables correspond directly to telemetry: the eight channels proposed in Review 2 (CPU and RAM utilisation, queue depth, uplink latency, jitter with packet loss, handover rate, residual energy, and temperature) and five trend channels that a telemetry agent can compute from its own recent readings. Four latent variables aggregate them into interpretable conditions—computational stress, network degradation, energy stress, and thermal stress—and the target, failure of the node within the horizon, is the child of those four conditions and of a reliability prior given by the device class. Table III lists the evidence of each condition.

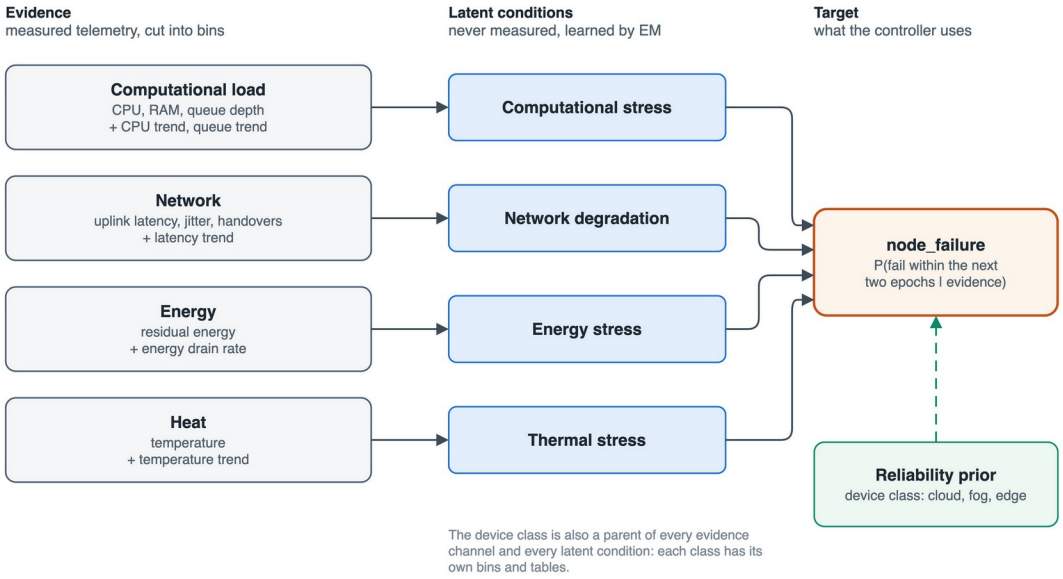


Fig. 2: Structure of the Bayesian Belief Network as implemented. Thirteen evidence channels feed four latent conditions, which with the device-class reliability prior feed node_failure. The device class is also a parent of every evidence channel and every latent condition, so each class has its own bins and tables. Evidence that is unavailable at a decision epoch is marginalised out rather than imputed.

TABLE III: Evidence Channels and the Latent Condition Each Feeds

Latent condition	Evidence channels	Source inside iFogSim2
Computational stress	cpu_util, ram_util, queue_depth, cpu_trend, queue_trend	work executed against host capacity; RAM provisioner; tuples resident in link and scheduler queues
Network degradation	uplink_latency, jitter_pktloss, handover_rate, latency_trend	link delay plus transmission load; variation of tuple inter-arrival times; topology changes over five epochs
Energy stress	residual_energy, energy_drain	power model against a declared energy budget; drain per epoch
Thermal stress	temperature, temp_trend	lumped thermal model driven by sustained utilisation (fog and cloud only; edge devices have no sensor)
Reliability prior	device class (cloud, fog, edge)	topology level

Two extensions to the Review 2 graph were adopted, each only after internal validation on the training scenarios showed a gain (Section VIII). The trend channels let the network see drift before it sees an extreme value, and conditioning on the device class lets a fog gateway and a battery-powered handset interpret the same reading differently. The network now has nineteen

variables—thirteen evidence channels, four latent conditions, the device class, and the target. Given the evidence actually observed at a decision epoch, the engine computes

$$P(F = fail \mid e) = \frac{\sum_{u} \prod_{V \in \mathcal{V}} P(V \mid pa(V))}{P(e)} \tag{1}$$

where $F \in \{\text{fail, not-fail}\}$ is the target variable and fail means that the node becomes unable to host its modules at some point within the next two epochs (200 simulation time units); e is the set of observed states of those channels that reported at this epoch; $\mathcal{V}$ denotes all nineteen variables, so that the product of local conditionals reconstitutes the joint distribution; u ranges over every joint assignment of the unobserved variables, namely the latent conditions plus any channel missing from e; and $P(e)$ normalises the two posterior probabilities to sum to one. The summation over u is what gives the network its tolerance of missing telemetry: a channel that failed to report widens the sum and blunts the posterior, but it does not prevent a decision, whereas a threshold on an unreported metric simply never fires.

Because each evidence channel feeds exactly one condition, the conditions are independent given the evidence and the device class c, and (1) factorises into a form that is both exact and cheap:

$$P(F = fail \mid e, c) = \sum_{\ell} P(F = fail \mid \ell, c) \prod_{k=1}^{4} P(L_k = \ell_k \mid e_k, c) \tag{2}$$

where ℓ ranges over the $3^4 = 81$ joint states of the four conditions and e_k is the evidence of condition k. A channel of e_k that did not report is summed out inside $P(L_k \mid e_k, c)$, weighted by its distribution for that device class. Vectorised over thousands of snapshots, inference costs about 3.6 µs per prediction.

Discretisation. Continuous channels are cut into at most eight ordered bins, separately for each device class with at least twenty training failures, at the cut points that maximise the mutual information between the binned channel and the failure label (Algorithm 1). Tertiles, which Review 2 proposed, put nearly every pre-failure reading into the same ‘high’ bin as a third of all healthy readings; supervised cuts become narrow where risk changes quickly and wide where nothing happens.

Tables. Before any failure data are seen, the latent tables are elicited as ranked nodes [15]: parent states map to the points 1/6, 1/2 and 5/6, a condition’s score is a weighted average of its parents’ points (computational stress, for example, weights CPU 0.45, RAM 0.35, and queue 0.20), and the failure table maps a weighted risk score to a probability between 2% and 88%. In the trained network the same form is learned: every (device class, parent, parent state) receives its own point θ, so that

$$P(L = \ell \mid x, c) = \Phi\left(\frac{b_{\ell+1} - s}{\sigma}\right) - \Phi\left(\frac{b_{\ell} - s}{\sigma}\right), \quad s = \sum_{i} \theta_{c,i,x_i} \tag{3}$$

where Φ is the standard normal distribution function, b_ℓ are the fixed boundaries of the condition’s three states, and σ is the elicited spread. A node with five eight-state parents then needs 40 numbers per class instead of 32,768 table rows, which is what makes fine bins learnable. A Gaussian penalty pulls each point toward its elicited value, and the failure table is learned with the elicited table as its Dirichlet prior (Algorithm 2). Online, once a report’s two-epoch horizon has passed, its outcome updates the failure table as a Dirichlet posterior around the offline one every five epochs; reports from nodes the controller acted upon are censored, because the action changed what would otherwise have happened.

Explanations. For each condition the engine computes how far the evidence lifted $P(L_k = \text{high} \mid e, c)$ above its prior for that device class, and names the condition with the largest lift as the principal cause—or none, when no condition rose. A condition about which the telemetry says nothing has zero lift and is never named, so an alarm reads, for instance, “P(fail) = 0.41, principally energy stress”.

C. Novelty 2: Risk-Aware Placement and Expected-Loss Migration

The posterior is not consumed as a binary alarm. Whenever a session needs a node—at start-up, after a mobility step, as a migration target, or during recovery—the controller scores each candidate node by

$$U(m, n) = \alpha_m\left(1 - \hat{L}(m, n)\right) + \beta_m\left(1 - \hat{E}(m, n)\right) + \gamma_m\left(1 - P(F_n = fail \mid e)\right) \tag{4}$$

where $\hat{L}(m, n)$ is the predicted end-to-end loop latency of session m on node n divided by the 150 ms deadline, estimated from the access delay and an M/M/1 queueing term for the node’s current load; $\hat{E}(m, n)$ is the access distance divided by 5 km, a proxy for radio energy; and the last term is the node’s reliability as the network currently estimates it. The weights follow the patient’s criticality: $(\alpha, \beta, \gamma) = (0.30, 0.10, 0.60)$ for a critical patient, (0.45, 0.15, 0.40) for a standard one, and (0.60, 0.25, 0.15) for a best-effort one. A critical patient is therefore steered toward a slower but safer node, whereas a best-effort patient takes the faster one. A user is moved for utility alone only if U improves by more than 0.10, so that sessions do not oscillate between near-identical nodes.

For migration, the framework replaces the fixed fault threshold of prior work with a cost-sensitive decision. A session on node n is migrated when

$$P\left(F_n = fail \mid e\right) \cdot C_{fail}(m) > C_{mig}(m,\ n \rightarrow n') \tag{5}$$

where $C_{\text{fail}}(m)$ is the criticality weight $\kappa(m)$ multiplied by the outage that a failure would cause at the session's current protection tier, and C_{mig} sums the stop-and-copy pause weighted by κ, the network cost of moving the analyser state (50 to 300 KB at 12.5 KB per ms), a fixed control cost, and any latency given up at the best alternative node n', the maximiser of (4). Rearranged, a session migrates once its node's failure probability exceeds its own threshold $P^{*} = C_{\text{mig}}/C_{\text{fail}}$. For an unprotected session these thresholds are 0.020 for a critical patient, 0.022 for a standard one, and 0.049 for a best-effort one: the fixed part of migration weighs more heavily against a smaller loss. There is no longer a single global threshold to choose, which answers the difficulty acknowledged by Rajab and Younis [4] and by Gupta and Singh [8].

D. Novelty 3: Criticality-Aware Selective Replication

Prior schemes provision a backup for every protected task [4], [6] or pre-select a backup node for every placed microservice [8]. The framework instead treats redundancy as a resource to be allocated where it changes the outcome. Each session is given a risk score

$$R(m) = P\left(F_n = fail \mid e\right) \cdot \kappa(m) \tag{6}$$

the expected consequence of losing it over the coming horizon, where $\kappa(m) \in [0, 1]$ is 1.0 for a critical patient, 0.5 for a standard one, and 0.15 for a best-effort one. Review 2 compared $R(m)$ with two fixed tier boundaries; the implementation removes those boundaries as well, choosing for every session the tier that minimises overhead plus expected loss,

$$\tau^{*}(m) = \arg\min_{\tau \in \mathcal{T}} \left[O(\tau,\ m) + R(m) \cdot D(\tau,\ m)\right] \tag{7}$$

where $\mathcal{T}$ = {none, checkpoint, warm standby}, $O(\tau, m)$ is the per-epoch overhead of the tier and $D(\tau, m)$ the outage a failure would cause under it. With no protection a failure costs a 30 ms heartbeat timeout plus a 300 ms cold restart; a periodic checkpoint every 50 ms reduces the outage to roughly 60–80 ms at the price of the state copies; a warm standby reduces it to 35 ms but also reserves a replica. For a critical patient the minimiser is no protection at $P = 0.001$, a checkpoint at $P = 0.05$, and a warm standby at $P = 0.30$. At most 20% of sessions may hold a warm standby at once; when more want one, the sessions of greatest risk keep it. Because the tiers are recomputed every epoch, standby capacity follows risk rather than being pinned at placement time.

E. End-to-End Workflow

The controller operates as a continuous loop over decision epochs of 100 time units, summarised in Fig. 3. It wakes one time unit after each epoch boundary, when every live device has reported:

1) Telemetry agents report their thirteen channels; a channel that did not report stays missing.
2) N1 computes the failure posterior of every fog node by (2) and records the principal cause.
3) Reports whose horizon has passed receive their outcome, and the online update refines the failure table.
4) N2 evaluates (5) for every running session and migrates those whose expected loss exceeds the cost of moving, to the node that maximises (4).
5) N3 recomputes (7) for every session, provisioning or releasing checkpoints and standbys within the 20% budget.
6) The decisions are actuated in the simulator; between epochs, a failed node is detected after the heartbeat timeout and each of its sessions recovers as its tier allows, re-placed by (4).
7) Failures, migrations, outages, and loop delays are recorded, and the next epoch begins.

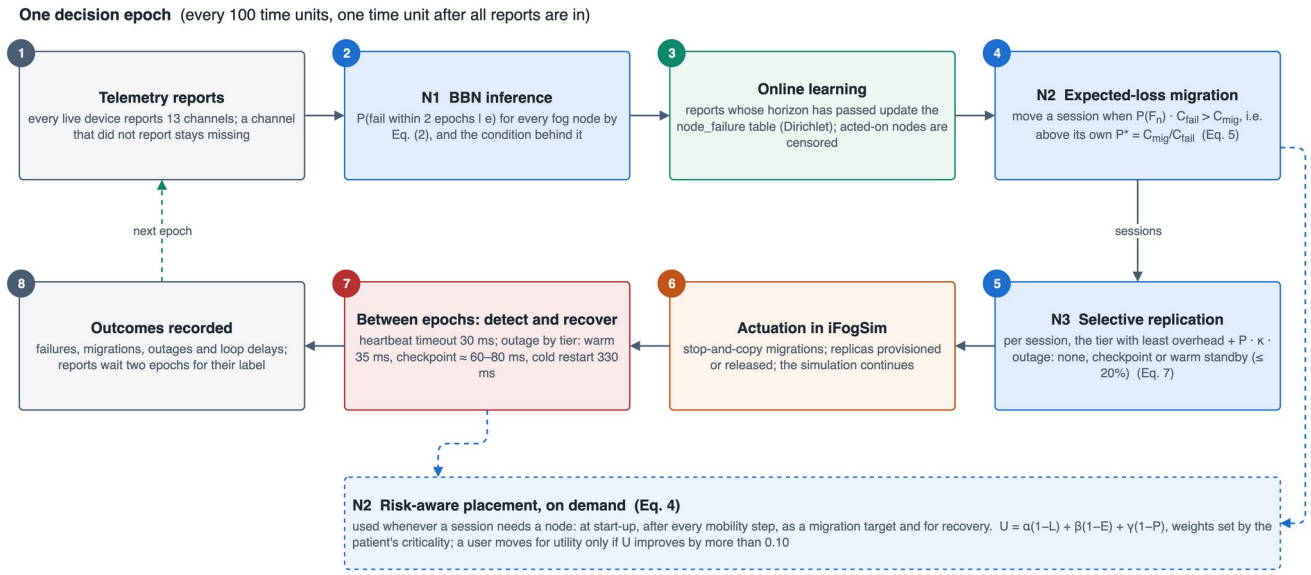


Fig. 3: Control loop of the framework over one decision epoch. Steps 1–3 perform inference and learning, steps 4–5 turn the posterior into migration and replication decisions, step 6 actuates them, step 7 detects and recovers failures between epochs, and step 8 records the outcomes that later label the reports. Risk-aware placement is invoked on demand wherever a session needs a node.

IV. Algorithms and Technologies Used

A. Algorithms

Three algorithms carry the framework. Algorithm 1 chooses the evidence bins, Algorithm 2 learns the network, and Algorithm 3 is the controller that runs inside the simulator.

Algorithm 1: Supervised per-class discretisation of one channel

Input: training values v and labels y of one channel for one device class; maximum bins $K = 8$; minimum bin share 0.1%
Output: strictly increasing cut points
$G \leftarrow$ 49 quantiles of v (2%–98%) $\cup$ {99, 99.5, 99.9}% quantiles $\cup$ 19 quantiles of the failing rows' values
Count failing and healthy rows between consecutive candidates of G; form cumulative sums
for every candidate interval $(g_i, g_j]$ **do** $\varphi(i, j) \leftarrow$ its term of the mutual information I(bin; y), with Jeffreys smoothing; $-\infty$ if it holds under 0.1% of rows
$M[0][0] \leftarrow 0$
for $k = 1 \ldots K$ **do**
    for $j = 1 \ldots |G| + 1$ **do**
        $M[k][j] \leftarrow \max_{i<j} \{ M[k-1][i] + \varphi(i, j) \}$; remember the arg max
$k^* \leftarrow$ the number of bins with the largest $M[k][|G| + 1]$
return the cut points recovered by backtracking from $M[k^*][|G| + 1]$

Because mutual information is a sum over bins, the dynamic programme of Algorithm 1 finds the optimal partition of the candidate grid exactly, which a unit test confirms against brute-force enumeration.

Algorithm 2: MAP expectation-maximisation with ranked-node latent tables

Input: training snapshots (binned evidence, device class, label); elicited network; Dirichlet strength 1.0; penalty $\lambda = 0.02$; 60 iterations
Merge identical (evidence, class, label) patterns into weighted rows *// 314,323 snapshots → 43,374 patterns*
Fit the evidence tables $P(x \mid c)$ from counts; initialise every ranked-node point θ at its elicited value
for $t = 1 \ldots 60$ **do**
    E-step: for every pattern, the posterior over the 81 latent configurations given its evidence *and* label, by (2); accumulate expected counts
    M-step (latent): for each condition and class, $\theta \leftarrow$ arg max of the expected log-likelihood of (3) $- \frac{1}{2} \lambda' \|\theta - \theta_0\|^2$, with $\lambda' = \lambda \times$ (expected count) / (number of points)
    M-step (target): $P(F \mid \ell, c) \leftarrow$ (expected counts + Dirichlet pseudo-counts from the elicited table), normalised
    if the mean log-likelihood changes by less than 10^{-7} **then** stop
return the network, its cut points, and its ranked-node points as **bbn_trained.json**

Algorithm 3: Per-epoch proactive fault-tolerance controller

Input: trained BBN; sessions with criticality κ; cost model; warm-standby budget 20%
at every epoch boundary + 1 time unit **do**
  for each live fog node n **do** p_n, $\text{cause}_n \leftarrow$ BBN(latest report of n) *// missing channels summed out*
  for each report whose horizon has elapsed and whose node was not acted on **do** update the Dirichlet counts; refresh every 5 epochs
  for each session m on node n **do**
    $n' \leftarrow \arg\max_{\text{feasible}} U(m, \cdot)$ *// (4)*
    if $p_n \cdot C_{\text{fail}}(m) > C_{\text{mig}}(m, n \to n')$ **then** migrate m to n' (stop-and-copy)
  for each session m **do** $\tau(m) \leftarrow$ (7); grant warm standbys to the riskiest sessions within the budget
on a node failure **do** after the heartbeat timeout, recover each session from its standby, checkpoint, or by cold restart on arg max U
on a mobility step **do** re-place the user by (4), with hysteresis 0.10

B. Technologies

Table IV lists the technologies used and the role each plays. The simulator side is written in Java against an unmodified iFogSim2 source tree [1], itself built on CloudSim [20]; the learning and evaluation side is written in Python, with scikit-learn [21] supplying the comparison models and pgmpy [22] serving as an independent check of the inference engine. All diagrams in this paper were drawn with draw.io.

TABLE IV: Technologies Used

Technology	Version	Role in the project
Java (OpenJDK)	21 (17+ supported)	the iFogSim2 extension: instrumented devices, fault model, controller, and Java BBN engine
iFogSim2 / CloudSim	upstream, unmodified	discrete-event simulation of the fog topology, application, mobility, and power
Python	3.12	dataset construction, discretisation, BBN training and inference, experiments, reports
NumPy, pandas, SciPy	current releases	vectorised exact inference and EM; data handling; ranked-node optimisation; Wilcoxon tests
scikit-learn	1.4+	random forest, gradient boosting, logistic regression, naive Bayes, and MLP baselines
pgmpy	1.0+	independent cross-check of the BBN posteriors
pytest	8+	50 unit and parity tests across inference, pipeline, and Java–Python agreement
Rich	13+	the nine-stage terminal demonstration
draw.io	desktop 31	every figure and chart in this paper and the presentation

V. Dataset Description

A. Sources

No public failure-trace dataset exists for a fog topology of this kind, so the dataset is generated by the instrumented simulator from three real inputs distributed with iFogSim2 [1]. The physical topology comes from the Melbourne central business district edge-resource dataset, whose 118 gateway sites, at their real coordinates, form the fog tier beneath a single cloud data centre. User movement comes from the Melbourne CBD mobility traces, along which 20 to 80 patients walk and hand over between gateways. The workload is a cardiac-monitoring application: a sensor sends vital signs to a client module on the patient's device, which forwards filtered readings to an ECG analyser on the serving fog node; the analyser returns an arrhythmia verdict that triggers the alarm actuator—the critical loop, with a 150 ms deadline—and sends a summary to the cloud for archival. Patients are 25% critical (intensive care), 45% standard, and 30% best-effort.

B. Fault Model and Labels

Because iFogSim2 has no failure model, one was added. Every fog node and edge device carries a hidden health in [0, 1] that loses, each epoch, an amount growing exponentially with a stress index that weighs CPU load, backlog, heat, energy depletion, link latency, jitter, and handovers, multiplied by log-normal noise; rare shocks remove 0.1–0.3 at once. Wear produces fail-slow symptoms [19] inside the simulator: with damage $d = 1 - \text{health}$, a node needs $(1 + 1.5d^2)$ times the CPU work per tuple, its links slow by a factor of $(1 + 0.6d^2)$, and it draws extra power and runs hotter. The telemetry measures these effects from iFogSim's own state; the network never sees the health. A node fails when its health reaches zero (wear-out), when a battery runs flat (edge devices), or suddenly, without precursor, which accounts for about one failure in five. A failed node drops every tuple and stops reporting. Each snapshot is labelled 1 if its node fails within the next 200 time units (two epochs) and 0 otherwise. Every random draw is keyed on (seed, scenario, node, epoch, stream), so different runs have independent failure luck, while two policies run on the same scenario and seed face identical luck.

C. Operating Regimes and Split

Twelve operating regimes vary the number of users and sensors, fog and edge capacity, backhaul, and power (Table V). Seven regimes train the model and five are held out, and the held-out runs also use seeds that training never saw (Fig. 4). Whole runs are held out, never random rows, because consecutive snapshots of one node are nearly identical and a row-level split would test on near-copies of training data. Each run lasts 10,000 time units, or 100 epochs; 31 telemetry runs were generated in all, and the closed-loop experiments add three further seeds of every held-out regime.

TABLE V: Operating Regimes of the Simulation

Id	Regime	What differs from the baseline	Split
0	baseline	40 users, 2,800 MIPS fog nodes	train
1	idle-fleet	20 users, one reading per 100 time units	held out
2	chatty-sensors	two sensors per user	train
3	thin-fog	half-size fog nodes	train
4	starved-fog	quarter-size fog nodes	train
5	fat-fog	double-size fog nodes, two sensors	train
6	weak-edge	slow, hot user devices	train
7	dense-edge	80 users	held out
8	slow-backhaul	250 ms fog uplinks with a fifth of the bandwidth	held out
9	fog-heavy	60 users, heavier analysis	held out
10	edge-heavy	heavier client module, hot devices	held out
11	saturated	thin fog, weak edge, slow backhaul	train



Fig. 4: Split of runs by operating regime and seed. Training uses seven regimes with seeds 101–103; held-out telemetry uses the five remaining regimes with seeds 201–202; the closed-loop experiments use the held-out regimes with seeds 201–205, so every run that evaluates the framework comes from a regime and a seed that training never saw.

D. Records and Statistics

One record is one device reporting at one epoch. Besides identifiers, it holds the thirteen channels of Table III, the device class, and the label; the hidden health and the failure time and cause are stored for analysis but are never features. An empty field means the channel did not report, which happens for temperature on edge devices, for jitter when fewer than three tuples arrived, for trends before an agent has history, and in the missing-telemetry experiments; nothing is imputed for the BBN. Table VI summarises the two pools. Cloud rows are kept but not scored, since the data centre failed in none of the 31 runs, which leaves 159,813 held-out fog and edge snapshots. A leak audit checks the split: of the nodes failing in both a held-out and a training run, 2.0% fail at the same epoch, which is chance level, against 92.1% under the Review 2 fault injector, whose random draws were shared across scenarios.

TABLE VI: Training and Held-Out Telemetry Pools

Pool	Runs	Snapshots	Positive	Prevalence	Failed nodes (wear-out / sudden / battery)
Training	21	314,323	1,508	0.48%	754 (486 / 125 / 143)
Held out	10	160,823	422	0.26%	211 (139 / 61 / 11)

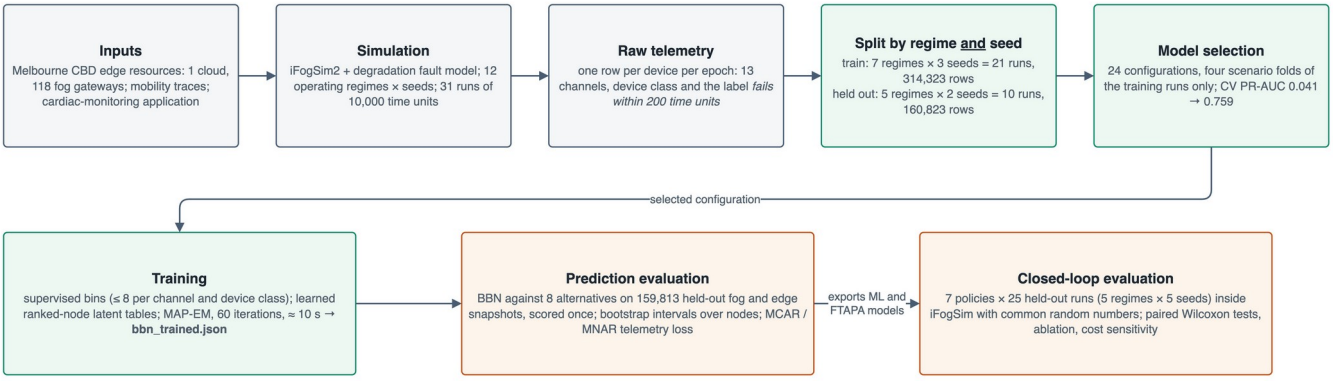


Fig. 5: Data and experiment pipeline. Telemetry generated by the instrumented simulator is split by regime and seed; every modelling choice is made on folds of the training runs; the trained network is then evaluated once offline and in the closed loop.

VI. Performance Metrics

Two families of metrics validate the framework: the quality of the failure prediction, measured offline on the held-out pool, and the quality of service delivered by the closed loop, measured inside the simulator.

A. Prediction Quality

With a failure prevalence of 0.26%, accuracy is meaningless—predicting ‘never fails’ scores 99.74%—so the evaluation relies on ranking and probability metrics. ROC-AUC is the probability that a randomly chosen failing snapshot is ranked above a randomly chosen healthy one. PR-AUC, the area under the precision-recall curve, is the stricter measure under heavy imbalance [23], because its chance level equals the prevalence, 0.0026. At an operating threshold chosen on training folds, alarms are scored by

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \tag{8}$$

Because the controller multiplies the posterior by a cost, the probabilities themselves must be accurate, not merely well ordered. The Brier score [24], the log-loss, and the expected calibration error (ECE) measure this over N snapshots with predictions p_i and labels y_i:

$$Brier = \frac{1}{N}\sum_{i=1}^{N}\left(p_i - y_i\right)^2 \tag{9}$$

$$LogLoss = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln p_i + \left(1 - y_i\right)\ln\left(1 - p_i\right)\right] \tag{10}$$

$$ECE = \sum_{b=1}^{K}\frac{\left|B_b\right|}{N}\left|\overline{y}_b - \overline{p}_b\right| \tag{11}$$

where the snapshots are sorted into $K = 15$ equal-mass bins B_b with mean label $\bar{y}_b$ and mean prediction $\bar{p}_b$. Early warning is the share of failing nodes flagged within the two epochs before they failed, with the mean lead time; robustness is the change in these metrics when channels are withheld at random (MCAR) or more often on degrading nodes (MNAR). Confidence intervals come from a bootstrap that resamples whole nodes [25], because snapshots of one node are not independent.

B. Service Quality

For a set of users $\mathcal{U}$ over a run of length T, each outage o of user u lasting d_o contributes to the downtime per user D and the availability A:

$$D = \frac{1}{|\mathcal{U}|}\sum_{u\in\mathcal{U}}\sum_{o\in\mathcal{O}_u} d_o, \qquad A = 1 - \frac{D\cdot|\mathcal{U}|}{|\mathcal{U}|\cdot T} \tag{12}$$

Downtime has three sources: node failures (detection plus recovery), the pauses of proactive migrations, and mobility handovers. Fault tolerance controls only the first two, so they are reported separately as *failures + migrations* downtime, the primary metric. Also reported are the users hit by a failure, the users evacuated before one, the 95th-percentile loop delay, the share of critical alarms that miss the 150 ms deadline, and the redundancy overhead as the share of sessions holding a warm standby. Policies are compared on the same runs with the paired Wilcoxon signed-rank test [26], which removes run-to-run variation and does not assume normality.

VII. Implementation

Fig. 6 shows how the implementation is organised. The simulator side is a single new package, org.fog.ft, of 31 Java source files added to an unmodified iFogSim2 checkout. InstrumentedFogDevice, a subclass of iFogSim's FogDevice, measures its true state at every resource-management interval, wears itself through FaultInjector, reports noisy telemetry with trends, and genuinely fails—dropping tuples and falling silent—when its health is exhausted. FaultToleranceController is a CloudSim entity that implements Algorithm 3; FtPolicy switches it between the seven policies of the comparison, and FtCostModel holds every time and cost constant. Predictors share one interface: BbnEngine performs exact inference from the trained model file and refines it online, while MlPredictor, HealthIndexPredictor, and OraclePredictor supply the comparison policies.

The Python side drives the simulator through parallel JVMs, one run per JVM because CloudSim keeps its clock in static state. build_dataset forms the pools and the leak audit; discretise, ranked_node, and bbn_fast implement Algorithm 1, (3), and Algorithm 2; select_bbn chose the configuration by four-fold cross-validation over the training scenarios; and the comparison and closed-loop experiments live in run_comparison and run_ft_experiments. Correctness is checked by 50 tests: 29 on inference (including that a missing channel is marginalised, which differs from imputing it, and that the named cause is never an unobserved condition), 18 on the pipeline (including that the dynamic programme matches brute force and that EM never decreases the likelihood), and 3 on Java–Python parity, which confirm that the two engines agree to within 10^{-15} on the same file. The Python engine also matches pgmpy [22] on both the elicited and the trained network.

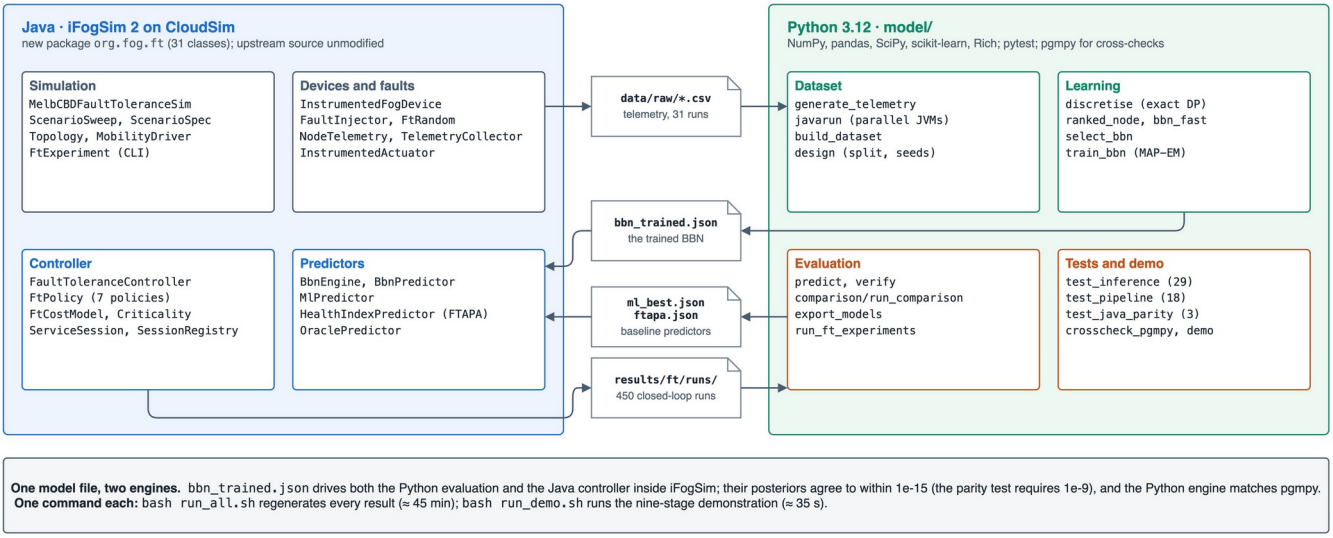


Fig. 6: Implementation. The Java package inside iFogSim2 (left) generates telemetry and runs the controller; the Python package (right) builds the dataset, learns and evaluates the BBN, and runs the experiments. They exchange the files in the centre, and one model file drives both inference engines.

Two scripts make the work reproducible: run_all.sh regenerates every dataset, model, table, and figure in about 45 minutes, and run_demo.sh presents the system in nine stages in about 35 seconds—building the extension, showing the topology and the data, training the BBN live, comparing predictors, running the trained network inside iFogSim with every migration and failure printed as it happens (Fig. 7), racing every policy on the same held-out run, and stating the verdict. In the run shown, the controller spent 2.24 ms per epoch.

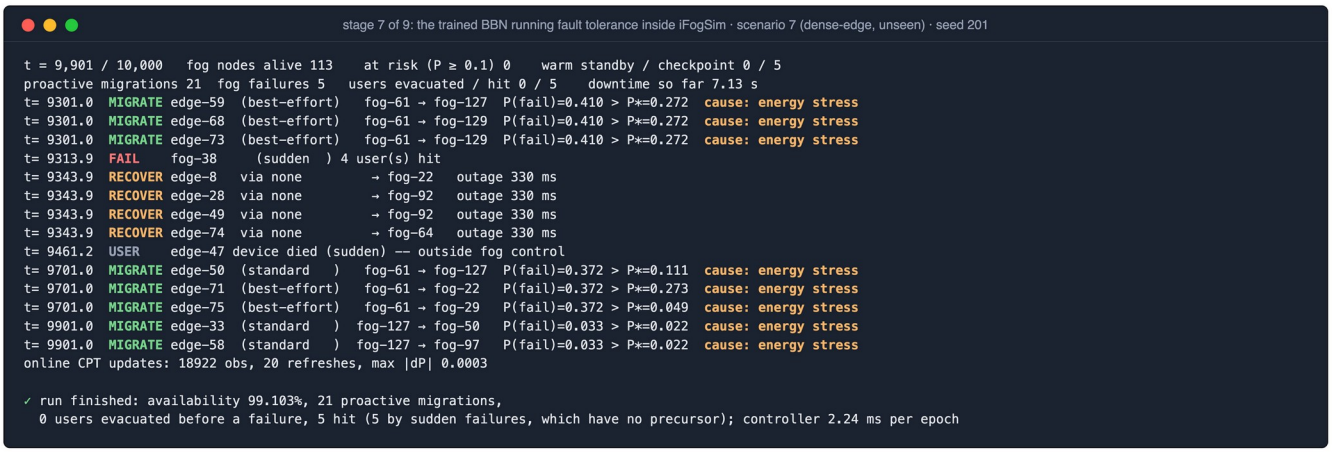


Fig. 7: Output of the live stage of the demonstration (scenario 7, seed 201, both unseen in training), redrawn from the terminal. Sessions on fog-61 are migrated once its failure probability passes each session's own threshold P*, with energy stress named as the cause; a sudden failure of fog-38, which no predictor can foresee, is detected and its four users recovered by cold restart.

VIII. Experiments and Results

All results come from held-out runs whose regimes and seeds were never used for training or for any modelling choice. The prediction experiments compare the BBN with eight alternatives on the held-out pool; the closed-loop experiments run seven policies on 25 held-out runs (five regimes × five seeds) with identical randomness.

A. Model Selection on the Training Scenarios

Twenty-four configurations were compared by four-fold cross-validation, each fold leaving out a group of training scenarios. Fig. 8 traces how each adopted change moved the cross-validated PR-AUC. The Review 2 graph with tertile bins reached only 0.041, little above the prevalence of about 0.005 in the training runs: tertiles discard almost all of the information in the tail where failures live. Supervised cut points raised it to 0.434, trend evidence to 0.562, learned ranked-node tables with eight bins to 0.653, bins chosen per device class to 0.753, and a weaker Dirichlet prior to 0.759, with the log-loss falling at every step. Only then was the held-out pool scored, once.

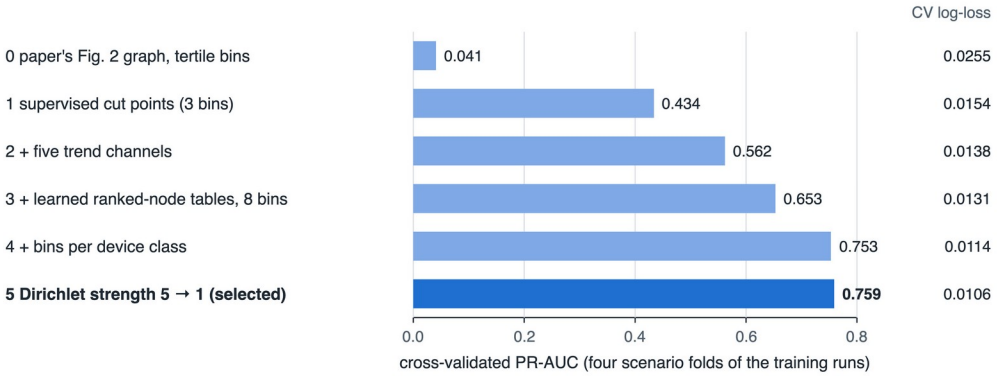


Fig. 8: Cross-validated PR-AUC and log-loss of the BBN on the training scenarios as each validated change was adopted.

B. Failure Prediction against Eight Alternatives

Table VII and Fig. 9 compare the BBN with a random forest [27], gradient boosting [28], naive Bayes, logistic regression, a multilayer perceptron, the FTAPA health index [8], a CPU threshold in the manner of [4], and the BBN with elicited tables only. Every choice a model makes—class weighting, alarm threshold—was made on out-of-fold predictions over the training runs. The classical models impute missing values with the training median and a missing-value indicator.

TABLE VII: Failure Prediction on 159,813 Held-Out Fog and Edge Snapshots (422 Failing)

Model	ROC-AUC [95% CI]	PR-AUC [95% CI]	Brier	ECE	F1	Flagged in time
BBN (proposed)	**0.866** [0.829, 0.895]	0.639 [0.585, 0.695]	**0.00128**	**0.00048**	0.658	0.63
Random forest	0.846 [0.802, 0.886]	**0.651** [0.592, 0.708]	0.00157	0.00844	**0.687**	**0.68**
Gradient boosting	0.851 [0.809, 0.886]	0.629 [0.569, 0.690]	0.00134	0.00064	0.645	0.63
Naive Bayes	0.862 [0.822, 0.895]	0.522 [0.465, 0.578]	0.01510	0.01689	0.549	0.62
Logistic regression	0.840 [0.803, 0.876]	0.469 [0.419, 0.521]	0.00180	0.00078	0.491	0.59
FTAPA health index	0.860 [0.822, 0.888]	0.376 [0.324, 0.433]	n/a	n/a	0.466	0.47
Neural network (MLP)	0.837 [0.799, 0.870]	0.239 [0.210, 0.282]	0.00328	0.00342	0.332	0.57
CPU threshold	0.861 [0.827, 0.888]	0.130 [0.114, 0.150]	n/a	n/a	0.295	0.53
BBN, elicited tables only	0.750 [0.717, 0.777]	0.015 [0.010, 0.021]	0.09975	0.29375	0.082	0.10

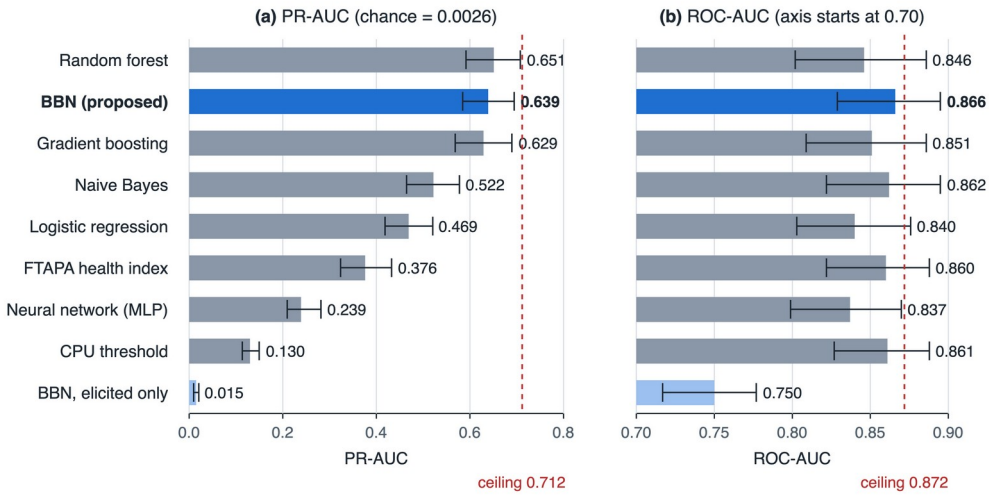


Fig. 9: Discrimination on the held-out pool with 95% bootstrap intervals over nodes. Dashed lines mark the ceiling that no predictor can exceed on this pool, because 29% of the failing snapshots belong to sudden failures.

The BBN has the highest ROC-AUC, 0.866, and the second-highest PR-AUC, 0.639, against 0.651 for the random forest; the two intervals overlap almost entirely. The random forest is ahead on F1 (0.687 against 0.658) and flags more failing nodes in time (68% against 63%). How close either comes to what is achievable is bounded by the data: 122 of the 422 failing snapshots belong to sudden failures, so even a predictor clairvoyant about every other failure would score at most 0.872 ROC-AUC and 0.712 PR-AUC. The BBN reaches 99% and 90% of that ceiling. On fog nodes it ranks failures as well as an oracle that knows each node's hidden health (0.898 against 0.896), which also serves as a leak check, since an honest model cannot beat that oracle.

Table VIII shows the alarms the BBN would raise at several thresholds. At the threshold chosen on training folds, 0.4994, precision is 0.825 and recall 0.547. Of the 211 failing nodes, 133 (63%) were flagged within the two epochs before they failed, with a mean lead of 127 time units—far more than the 5–30 units a live migration needs—at a cost of 0.3 false alarms per 1,000 healthy snapshots. The controller, however, uses no fixed threshold at all; each session's threshold follows from (5).

TABLE VIII: BBN Alarms on the Held-Out Pool at Different Thresholds

Threshold	TP	FP	FN	Precision	Recall	F1
0.02	297	761	125	0.281	0.704	0.401
0.05	292	397	130	0.424	0.692	0.526
0.10	286	247	136	0.537	0.678	0.599
0.20	276	139	146	0.665	0.654	0.659
0.30	268	87	154	0.755	0.635	0.690
0.50	231	49	191	0.825	0.547	0.658

C. Calibration

The BBN's probabilities are the most trustworthy of all the models: Brier score 0.00128, log-loss 0.00815, and ECE 0.00048, with a mean predicted probability of 0.0030 against an observed failure rate of 0.0026. Fig. 10 shows why this matters. The random forest ranks almost as well, but it assigns about 0.04 to 22,476 snapshots that fail at a rate of 0.0006; multiplied by a failure cost, such probabilities would trigger migrations that the evidence does not justify. The BBN is not perfect either—between 0.01 and 0.3 it over-predicts by about a factor of two—but its error is an order of magnitude smaller.

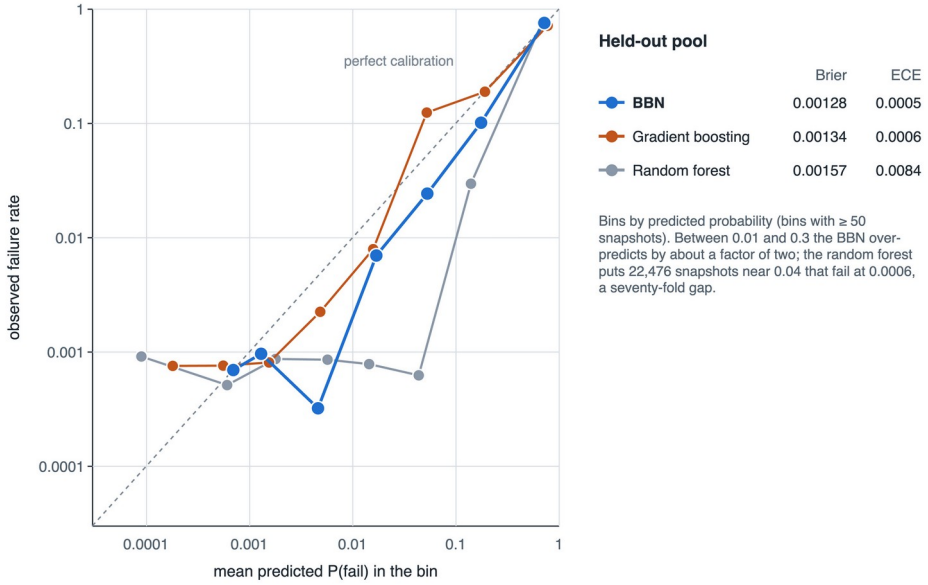


Fig. 10: Reliability diagram on the held-out pool (log scales). Each point is a bin of snapshots by predicted probability; points on the diagonal are perfectly calibrated.

D. Partial Evidence and Missing Telemetry

The BBN produced a prediction for every one of the 159,813 snapshots however many channels were dark. Withholding single channels shows where its information comes from (Table IX): the energy drain rate matters most, because in this simulator a worn node draws extra power before its other symptoms grow, followed by temperature, uplink latency, and residual energy. With six channels removed at random, the ROC-AUC barely moves (0.863) while the PR-AUC falls to 0.521.

TABLE IX: Effect of Withholding One Channel from the BBN at Inference

Channel withheld	PR-AUC	ROC-AUC	Change in PR-AUC
none	0.639	0.866	—
energy_drain	0.532	0.866	−0.107
temperature	0.592	0.866	−0.047
uplink_latency	0.593	0.871	−0.046
residual_energy	0.603	0.865	−0.036
any other single channel	0.639–0.644	0.866–0.868	within ±0.005

Fig. 11 withholds channels more often on degrading nodes, the realistic case in which a telemetry agent falls silent because its node is in trouble. Ranking degrades for every model; the BBN and the random forest stay close, while gradient boosting, logistic regression, and the FTAPA index fall away quickly. In probability quality, the BBN keeps the lowest log-loss at every loss rate, averaging 0.0171 over 30–60% loss against 0.0189 for gradient boosting. For pure ranking under heavy loss, naive Bayes—also a Bayesian network—holds up best (ROC-AUC 0.821 against the BBN's 0.700). When half or more of the channels are missing on failing nodes, every model falls toward the prior.

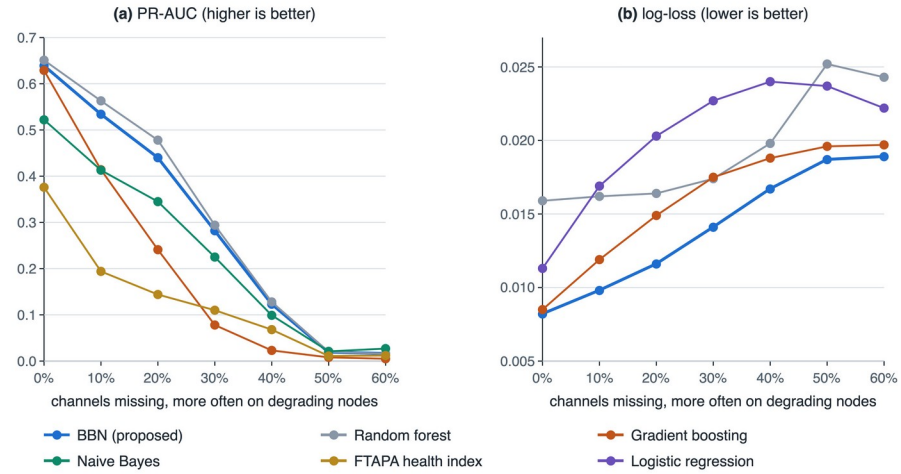


Fig. 11: Robustness when telemetry channels go missing more often on degrading nodes (MNAR): (a) PR-AUC and (b) log-loss on the held-out pool. Models were trained on complete telemetry; the BBN sums missing channels out, the others impute them.

E. Explanations and Data Efficiency

Table X checks whether the condition the BBN names matches how a node actually failed. Battery failures are blamed on energy stress in 86% of positive snapshots. Wear-out failures are attributed to energy stress in 92%, which is the expected answer in this simulator, since a worn node's extra power draw is its earliest symptom. For sudden failures, which have no precursor, the network raises no condition at all in 52% of cases. The elicited network, which uses no failure data whatsoever, already reaches a ROC-AUC of 0.750; a classifier cannot be trained at all without labels. Trained on a single run, the BBN reaches a PR-AUC of 0.552, close to the random forest's 0.570.

TABLE X: Principal Cause Named by the BBN against the Actual Failure Cause (Share of Positive Snapshots)

Actual cause	Snapshots	Computational	Energy	Network	Thermal	None
Battery	22	0.00	**0.86**	0.14	0.00	0.00
Wear-out	278	0.04	**0.92**	0.04	0.00	0.00
Sudden	122	0.02	0.16	0.30	0.00	**0.52**

F. Closed-Loop Fault Tolerance

Table XI and Fig. 12 report the seven policies on the same 25 held-out runs. No fault tolerance places users on the nearest gateway and cold-starts them after a failure; reactive checkpointing follows Rajab and Younis [4]; threshold migration follows FTAPA [8], with a tuned health-index threshold and a warm backup for every session; the ML policy runs the proposed controller with gradient boosting, the best classical model out-of-fold, in place of the BBN; and the oracle runs it with a predictor that knows each node's hidden health and wear law, bounding what any predictor could achieve.

TABLE XI: Closed-Loop Results over 25 Held-Out Runs (Mean per Run)

Policy	Failures + migrations / user (ms)	All downtime / user (ms)	Users hit	Warm standby	Loop p95 (ms)	Critical deadline misses
No fault tolerance	517.2	603.4	78.5	0.0%	116	3.82%
Cloud-only placement	2.3	2.3	2.0	0.0%	310	100.00%
Reactive checkpoint + backup [4]	110.9	201.2	89.2	0.0%	122	4.39%
Threshold migration, FTAPA [8]	37.7	132.4	54.1	100.0%	148	5.46%
Expected-loss controller + ML	18.8	86.2	3.8	0.3%	35	0.22%
Proposed BBN framework	**20.0**	**87.6**	**4.5**	**0.2%**	**35**	**0.13%**
Oracle predictor (upper bound)	14.5	81.5	3.4	0.4%	36	0.12%

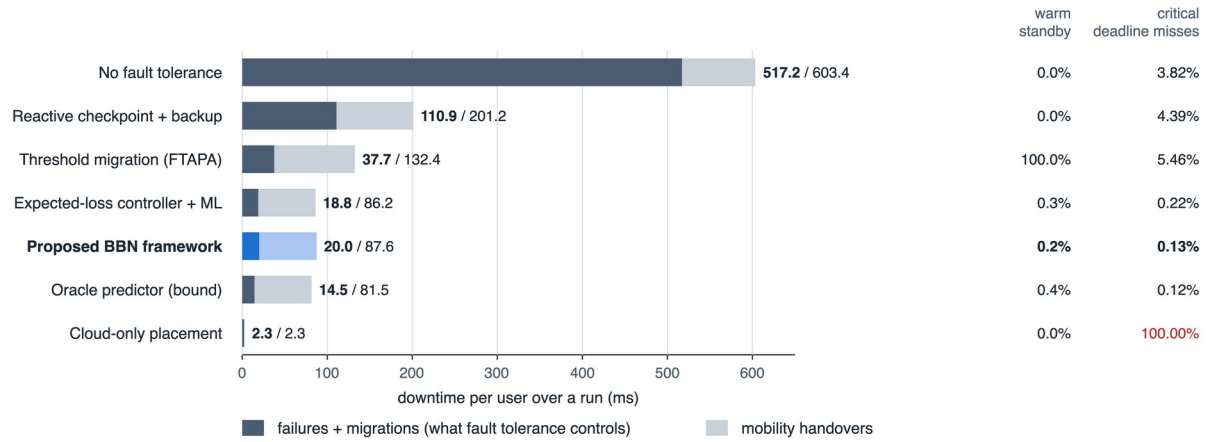


Fig. 12: Downtime per user over a run, split into the part fault tolerance controls (failures and migration pauses) and mobility handovers, with the share of sessions holding a warm standby and of critical alarms that missed the 150 ms deadline.

Against no fault tolerance, reactive checkpointing, and FTAPA, the BBN framework reduces failures-plus-migrations downtime by 96.8%, 85.0%, and 55.4% and critical patients' downtime by 82.3%, 58.1%, and 39.3% (Table XII). FTAPA buys its protection with a warm standby for every session; the BBN framework holds one for 0.2%. The number of users hit by a failure falls from 54.1 under FTAPA to 4.5 per run; 1.4 of those are hit by sudden failures, which every policy suffers to the same degree (1.1 to 1.4 users per run) because no predictor can see them coming. Cloud-only placement has the least downtime of all, since there is no fog state to lose, but every alarm crosses the wide-area network: its 95th-percentile loop delay of 310 ms misses the 150 ms deadline on every critical alarm.

TABLE XII: Paired Comparison of the BBN Framework with Each Policy (Wilcoxon Signed-Rank, 25 Runs)

Against	Failures + migrations change	p-value	Critical downtime change	p-value
No fault tolerance	−96.8%	<0.0001	−82.3%	<0.0001
Reactive checkpoint + backup	−85.0%	<0.0001	−58.1%	<0.0001
Threshold migration, FTAPA	−55.4%	0.0016	−39.3%	<0.0001
Expected-loss controller + ML	+18.5%	0.1219	−1.3%	0.9368
Oracle predictor	+58.3%	0.0009	+3.1%	0.0883

With complete telemetry the BBN-driven and ML-driven controllers are statistically indistinguishable (+18.5%, $p = 0.12$). Relative to the oracle, the BBN framework removes 98.9% of the fault-related downtime that the oracle removes over no fault tolerance (20.0 against 14.5 ms per user).

G. Degraded Telemetry in the Closed Loop

The three telemetry-driven policies were rerun with 30% of channels missing, more often on degrading nodes (Table XIII). The BBN framework now has 29.2% less fault-related downtime than the ML-driven controller ($p = 0.006$) and 17.9% less total downtime ($p = 0.024$): when a degrading node's readings go dark, a model that imputes a healthy-looking median is misled, whereas one that sums the channel out is merely less certain. FTAPA, with a warm backup for every session, is the most robust on fault-related downtime (33.9 against 65.9 ms, $p = 0.004$), although total downtime is equal ($p = 0.56$); its robustness is bought with standby capacity for every session.

TABLE XIII: Closed Loop with 30% of Channels Missing on Degrading Nodes (MNAR)

Policy	Failures + migrations / user (ms)	All downtime / user (ms)	Change vs complete telemetry	Users hit
Threshold migration, FTAPA	33.9	133.7	−10.2%	41.4
Expected-loss controller + ML	90.6	156.2	+381.5%	15.6
Proposed BBN framework	**65.9**	**131.8**	+229.4%	**13.4**

H. Ablation and Cost Sensitivity

Fig. 13 removes one component at a time. Without the risk term in placement, fault-related downtime rises by 71.0% ($p = 0.004$); without selective replication, by 57.7% ($p = 0.020$). Removing online learning (+10.9%, $p = 0.63$) or proactive migration (−7.2%, $p = 0.12$) makes no significant difference: once placement already keeps sessions away from fragile nodes and replication protects the ones that remain, the pause of a migration costs about as much as the failures it averts. Fig. 14 multiplies every migration cost by factors from 0.25 to 4. The number of migrations falls from 104.1 to 10.8 per run, yet fault-related downtime stays between 20.0 and 23.3 ms per user: the self-calibrating rule trades migrations for risk without needing a tuned threshold.

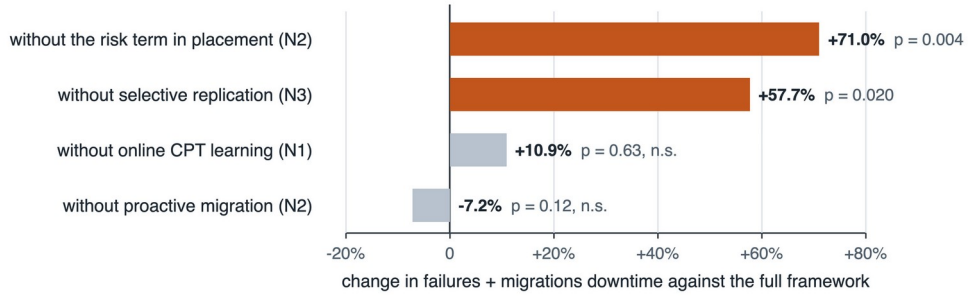


Fig. 13: Ablation: change in failures-plus-migrations downtime when one component of the framework is removed (paired Wilcoxon p-values over 25 runs).

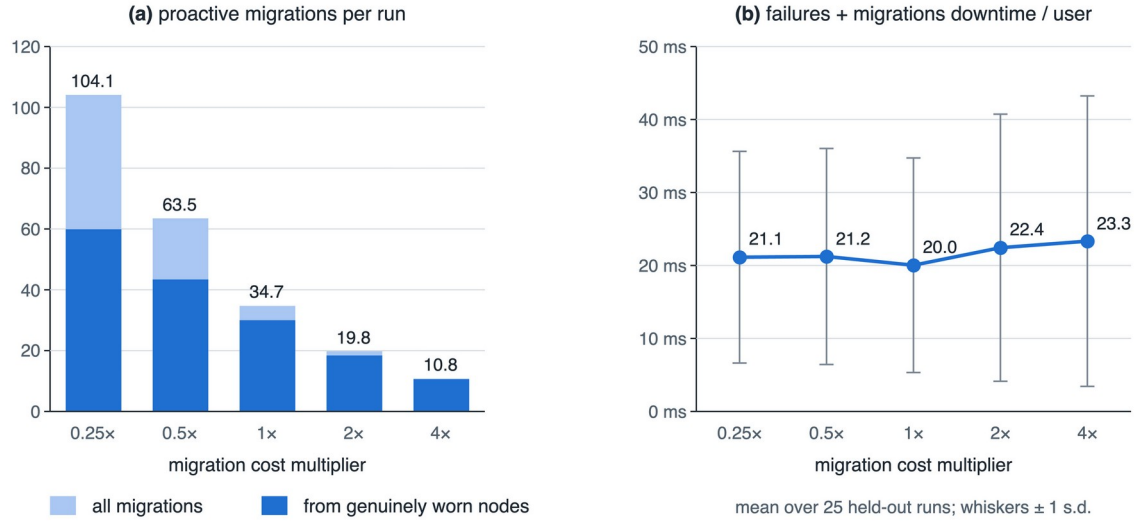


Fig. 14: Sensitivity to the migration cost: (a) migrations per run, of which those from genuinely worn nodes in dark blue, and (b) failures-plus-migrations downtime per user.

IX. Inferences

The experiments support seven inferences, including several that qualify the case for the BBN.

1) **A BBN can match strong black-box classifiers at ranking failures.** Its ROC-AUC is the highest of nine models and its PR-AUC is statistically tied with the random forest's; both sit at 99% and 90% of the ceiling the data allow. The random forest is nevertheless slightly ahead on PR-AUC, F1, and early detection.
2) **Its decisive advantage is calibration.** The BBN has the best Brier score, ECE, and log-loss, and its log-loss stays best at every level of telemetry loss. Since every decision rule multiplies the posterior by a cost, this is the property that matters most for control.
3) **Summing out missing telemetry beats imputing it when channels fail with their nodes.** With complete telemetry the BBN- and ML-driven controllers are equivalent; with 30% degradation-correlated loss the BBN-driven one has 29% less fault-related downtime. Under heavy loss, however, no model ranks well, and naive Bayes ranks best.
4) **Probabilistic, criticality-aware decisions outperform thresholds and uniform redundancy.** Against FTAPA the framework cuts fault-related downtime by 55% and critical downtime by 39% while holding a standby for 0.2% of sessions instead of all of them. FTAPA's blanket standby does make it the more robust under degraded telemetry.

5) **Most of the gain comes from where sessions are placed and how they are protected.** The ablation shows risk-aware placement and selective replication to be significant and proactive migration not, so the value of the posterior lies in steering placement and redundancy as much as in triggering moves.
6) **The expected-loss rule removes the threshold-selection problem.** A sixteen-fold change in migration cost changes the number of migrations tenfold but leaves downtime within 20–23 ms per user.
7) **The limits are real.** Sudden failures, 29% of the held-out positives, cannot be predicted and are mitigated only by redundancy. The fault model shapes what can be learned—energy drain is the earliest symptom here, but other hardware would show other precursors and require retraining. And every result comes from simulation under an accelerated-wear model, not from field data.

X. Conclusion and Future Work

This paper argued that fault tolerance in fog computing remains predominantly reactive, and that the proactive mechanisms so far reported reduce an inherently uncertain judgement to a fixed threshold on a single metric. It presented, implemented, and evaluated a framework in which a causal Bayesian Belief Network infers the posterior probability of fog-node failure from noisy and possibly incomplete telemetry, an expected-loss rule and a risk-aware utility turn that posterior into migration and placement decisions with no global threshold, and a criticality-aware scheme spends standby capacity where risk and consequence warrant it. Built inside iFogSim2 and evaluated on operating regimes and seeds withheld from training, the network ranks failures as well as the strongest of eight alternatives while producing the best-calibrated probabilities, and the framework reduces fault-related downtime by 97% against no fault tolerance, 85% against reactive checkpointing, and 55% against threshold migration, while reserving warm standbys for 0.2% of sessions. Its advantage over an ML-driven controller appears precisely when telemetry fails alongside the nodes it describes.

Three directions follow. First, the evaluation should move from a synthetic degradation model to real failure traces from fog or edge deployments, which would test whether the structure and the learned precursors transfer. Second, a dynamic Bayesian network could model the temporal evolution of degradation explicitly instead of through trend channels, and could extend the useful horizon. Third, inference could be distributed across clustered fog nodes, removing the controller as a single point of failure and allowing the framework to scale to larger deployments.

References

[1] R. Mahmud, S. Pallewatta, M. Goudarzi, and R. Buyya, “iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments,” *Journal of Systems and Software*, vol. 190, p. 111351, 2022.

[2] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, “iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments,” *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.

[3] D. A. Hasan and S. R. M. Zeebaree, “Proactive fault tolerance in distributed cloud systems: A review of predictive and preventive techniques,” *Indonesian Journal of Computer Science*, vol. 13, no. 2, 2024.

[4] H. T. Rajab and M. F. Younis, “Dynamic fault tolerance aware scheduling for healthcare system on fog computing,” *Iraqi Journal of Science*, pp. 308–318, 2021.

[5] S. A. Syed, M. Rashid, S. Hussain, F. Azim, H. Zahid, A. Umer, A. Waheed, M. Zareei, and C. Vargas-Rosales, “QoS aware and fault tolerance based software-defined vehicular networks using cloud-fog computing,” *Sensors*, vol. 22, no. 1, p. 401, 2022.

[6] M. H. Naim, J. M. Zain, and K. A. Jalil, “Fault tolerance mechanism for software application through fog computing as middleware,” *International Journal of Computing and Digital Systems*, vol. 5, no. 3, 2016.

[7] A. Reyana, S. Kautish, K. A. Alnowibet, H. M. Zawbaa, and A. W. Mohamed, “Opportunities of IoT in fog computing for high fault tolerance and sustainable energy optimization,” *Sustainability*, vol. 15, no. 11, p. 8702, 2023.

[8] S. Gupta and N. Singh, “Optimized fault-tolerant placement of microservices in distributed fog networks,” *International Journal of Data Informatics and Intelligent Computing*, vol. 4, no. 4, pp. 52–69, 2025.

[9] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA: Morgan Kaufmann, 1988.

[10] N. Fenton, M. Neil, and D. Marquez, “Using Bayesian networks to predict software defects and reliability,” in *Proc. 5th Int. Conf. Mathematical Methods in Reliability (MMR07)*, Glasgow, UK, 2007.

[11] P. Weber, G. Medina-Oliva, C. Simon, and B. Iung, “Overview on Bayesian networks applications for dependability, risk analysis and maintenance areas,” *Engineering Applications of Artificial Intelligence*, vol. 25, no. 4, pp. 671–682, 2012.

[12] P. A. Aguilera, A. Fernández, R. Fernández, R. Rumí, and A. Salmerón, “Bayesian networks in environmental modelling,” *Environmental Modelling & Software*, vol. 26, no. 12, pp. 1376–1388, 2011.

[13] B. Pang and E. S. Abramov, “Reliability analysis and parameter selection for IoT communication based on deep learning,” *Eng*, vol. 6, no. 8, p. 171, 2025.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[15] N. E. Fenton, M. Neil, and J. G. Caballero, “Using ranked nodes to model qualitative judgments in Bayesian networks,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 10, pp. 1420–1432, 2007.

[16] A. P. Dempster, N. M. Laird, and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[17] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. Cambridge, MA: MIT Press, 2009.

[18] U. M. Fayyad and K. B. Irani, “Multi-interval discretization of continuous-valued attributes for classification learning,” in *Proc. 13th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 1993, pp. 1022–1027.

[19] H. S. Gunawi *et al.*, “Fail-slow at scale: Evidence of hardware performance faults in large production systems,” in *Proc. 16th USENIX Conf. File and Storage Technologies (FAST)*, 2018, pp. 1–14.

[20] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, “CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms,” *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.

[21] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[22] A. Ankan and A. Panda, “pgmpy: Probabilistic graphical models using Python,” in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 6–11.

[23] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLoS ONE*, vol. 10, no. 3, p. e0130432, 2015.

[24] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[25] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[26] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[27] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[28] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.